Green Cycle - Engineering Management

Authors: *Boris Marinković, Janko Kondić, Nikola Gostovikj*
Date: *May 5, 2026*

1. Antipatterns

1.1. Treat Your Team Like Children

This antipattern refers to micromanaging, assigning tasks without explanation, not trusting people to make decisions, and expecting everyone to just execute orders without thinking.

In Green Cycle, each team member has a clearly defined role (for complete explanation read *Green Cycle - Scrum Basics*). We deliberately avoided situations where one person reviews and approves every decision made by another. For example, when one team member was building the React components, he made his own decisions on structure and naming conventions without needing sign-off on each file. The Definition of Done requires a code review by at least one other member, but that is for quality, not control. The distinction matters.

1.2. Ignore Low Performers

Ignoring someone who is struggling is the worst thing a manager can do. It hurts the team's output and is unfair to the person, who may just need direction.

In our case, we are a three-person team, so there is no real hierarchy, but the dynamic still applies. If one of us is behind on a sprint task, the Scrum Master is responsible for catching this early. We have weekly check-ins where everyone shares what they have completed and what is blocking them. This is intentionally modeled to follow the coaching approach as much as possible: set a timeframe, set specific goals, check in regularly. We do not wait until the sprint review to find out something was not done**.**

1.3. Compromise the Hiring Bar

This antipattern is about accepting people into a team who are not a good fit, usually because you need the headcount quickly. In academic context, it translates to not taking shortcuts when assigning responsibilities. Giving a critical task to someone who is not equipped for it, just to close the sprint backlog faster, is the same mistake.

During Sprint 0 we assigned roles based on actual skills of the team members. We did not split tasks just to make things look equal on paper. An evenly split task list would look fair but would have introduced unnecessary risk in a 4-month project.

2. Positive Patterns

2.1. Set Clear Goals

This is the most important thing a good leader does. Vague goals lead to wasted effort and frustration.

Our project is structured around four milestones from the Green Cycle - *Project Charter* document, each tied to a sprint with a concrete deadline. Within each sprint, every task in the backlog has an assigned owner, a story point estimate, and a reference to the user story it covers. We use the Fibonacci scale for estimation to force honest conversations about complexity. When a task is ambiguous, we break it down further before assigning it. A task like "implement search" would never appear in our sprint backlog as-is, it becomes separate tickets for the backend filter endpoint, the frontend filter UI, and the integration.

2.2. Be Honest

Related to the above, a good leader does not hide problems or give false reassurance. This applies to us in how we document and report on the project.

In our *Green Cycle - Hypothesis Testing* document, we explicitly marked one hypothesis as only partially confirmed because the interviewee raised a concern we could not fully resolve. We did not round it up to “confirmed” status just to make the results look cleaner. The same applies to the *Green Cycle - Usability Testing* document, where we listed the specific tasks that caused problems and the specific UI issues that need fixing, rather than writing a summary that makes everything sound fine. The Project Charter also includes a risk table with honest likelihood and impact ratings.

2.3. Remove Roadblocks

A good team leader does not just track tasks, they actively remove whatever is blocking progress. This was the pattern we found most practically useful. In Sprint 1, one of the early blockers was tooling: we needed to decide on a Scrum board before we could start tracking work. Rather than leaving everyone waiting, the Scrum Master evaluated the options and proposed GitHub Projects as the solution, specifically because we already had the repository there. The decision was made quickly and it unblocked the rest of the team. This is a small example, but the pattern is the right one: the Scrum Master's job is not to report on blockers, it is to eliminate them.